As we have seen in section 12.3, in DBSCAN the clustering is computed by exploring and leveraging the relations between points, that are therefore interconnected: sharding a dataset would change the ε-neighborhood of most points, and some of the core points might not be recognized as such because their ε-neighborhood is scattered across several shards.

Although it is possible to think about a MapReduce job counting the size of the ε-neighborhood of each point with a distributed computing model, the core of DBSCAN relies on sequentially going through points and their neighbors, and thus it seems it would make more sense to explore different options.

We have already discovered in section 13.1.4 that we could use canopy clustering as a first step, apply DBSCAN to each canopy separately, and then iteratively merge clusters in neighboring canopies when points close to their borders, or in the overlapping regions, are within each other's ε-neighborhood.

This approach becomes problematic for a few reasons:

- It's hard to keep track of the canopies that needs to be checked, and of the points inside the canopies to compare: all pair of canopies need to be compared to check if they overlap or their distance is smaller than ε, and then for each pair of canopies all points need to be compared to the other canopy;
- Given the shape of the envelopes (hyperspheres), it is complicate to compute the distance between points in a canopies' external rings and the other canopy (see section 10.4.3 and figure 10.23 to get an idea of the